

# F20FO/F21FO - Digital Forensics

## Lab 08: Live Forensics and IR

### Lab 8: Objectives

This lab intends to introduce you to concepts of digital forensic imaging and analysis. At the end of the lab you will be able to use:

1. VM and Autopsy
2. Procmon
3. Live forensics

### Notes

There is nothing to submit at the end of this lab, however students are encouraged to log their results in a document. There are some questions **coloured in blue** that you can use as indication to record in your logbook (e.g. lab08\_logbook). Often the lab instructions are intentionally open ended, and you will have to figure some things out for yourselves.

The lab will require you to install software and download additional resources from the lab08\_reources folder on Canvas and online. Changes to your machine should not impact on its functionality - however you are recommended to use a VM. You need to select the appropriate VM platform (e.g. Linux or Windows for this lab).

This lab will cover several important fundamental concepts for forensics and security, and will also look at introducing you to live forensics.

### Lab 8.1: VM and Autopsy

Throughout your career in forensics you will encounter new software, operating systems or data that you just don't know what to do with. This lab will begin with setting up an experiment environment.

To make life easy we are going to use Virtual Machines (VMs). Here we explore some aspects of VMs which can be useful in forensic investigations.

You should have VirtualBox or VMWare already installed, if you do not have any of these VMs installed, you should refer to the previous lab sessions for this course.

An important item you will use both within Forensics and other areas of computing are ISO files. For this exercise we are going to use a 'light' version of Linux (Lubuntu); it is based on Ubuntu. Download the ISO (select 24.04.3 LTS (Noble Numbat)) from:

<https://lubuntu.me/downloads/>

Click on 'Desktop 64-bit' and then select the 24.04.3 iso file.

Launch your virtualisation software and create a virtual machine using the Lubuntu ISO.

How much RAM is allocated to your VM?

Start the VM. Run through the installation of Linux [you will need to Start Lubuntu first, then on the desktop is an option to install]

You will need to create a user account. Remember your username and password!

Once fully installed reboot the VM, you will then be able to login.

Create a file on the desktop - use your imagination. We are going to look for this file shortly - so try not to make it too obscure!

Shutdown the VM.

### Lab 8.1.1: Viewing a VM

VMs need to be treated like any other piece of digital evidence, in that we need to maintain their integrity and any process conducted must be repeatable by a third party to confirm our findings. However, as discussed earlier in the course, VMs are nice in that they are portable and can be copied and analysed - and we can snapshot the state of the system at any point.

We can also use VMs as a primitive "forensic image" and most of the forensic tools are able to read VMs as though they are forensic images. Although the VM will need to be turned off, or at least powered down

You used Autopsy in the previous lab (Lab07). If you don't already have it installed, download and install Autopsy (Follow Lab07 instructions).

Launch Autopsy.

Create a new case (select a name and location for the case)

Select Disk Image or VM File

If you are using VMWare, browse to your VM folder and select the .vmdk file.

If you are using VirtualBox, you need to convert the .vdi file to a raw format.

- Use 'vboxmanage', a command line tool, which is located in the directory where VirtualBox is located.
- An example of how VboxManage can be used:

```
vboxmanage clonehd "path and filename of diskfile.vdi" "path and filename of outputimage.img --
format raw
```

- Note that you need to research how to use VboxManage for your platform (e.g. Linux/Windows). More info on VboxManage, <https://www.virtualbox.org/manual/ch08.html>

In Autopsy, once the Image/VM file is loaded, Click Next - leave the default options.

Click Next.

Autopsy will then work its magic and add the evidence.

Note that you can't add the VM if it is still running.

Using the Data Sources tree view can you find the data you put on the desktop? Upload a screenshot of your desktop in the quiz.

Most forensic tools view the virtual disk file (VMDK) as an image and allows it to be added to a case. You can now browse your VM as if it was normal evidence.

## Lab 8.2: Procmon

This is a good exercise to get you thinking about forensics and analysis - similar techniques are used when doing malware analysis. Procmon is capable of monitoring changes to the registry, file system changes, network activity, and process and thread activity. This was covered in the live forensics lecture. This section of the lab provides better results on a Windows system.

To identify what a program (or file) does, it is useful to monitor the changes it makes to a computer system when it is installed, when it launches, when it stores data or when settings are modified. By being able to identify what changes have occurred you can be more confident in the conclusions you draw whilst conducting an analysis. You will use similar techniques when investigating malware.

Procmon.exe (from SysInternals can be downloaded from <https://docs.microsoft.com/en-us/sysinternals/downloads/procmon>). It is capable of monitoring changes to the registry, file system changes and process and thread activity.

In the following examples you will identify where Internet Options stores the proxy settings. Learn from these examples and get in the habit of quickly setting-up Process Monitor to monitor changes to your system. [other tools exist, process monitor is a relatively simple interface]

### Lab 8.2.1: Change to Internet Options

Download and extract Procmon.exe from the supplied link. Record the name of the folder where you store it. There is also a useful Help System (procmon.chm) within the zip file.

Run Process Monitor.

When you first launch Process Monitor it begins to capture events, an event is a change to the system.

- To stop and start event capture, type Ctrl + E
- To clear the screen of all the captured logs type Ctrl + X.
- Explore other menu options. You need an enquiring mind to get the most out of the labs and to develop as a digital “expert”!

You will notice that the application captures thousands of events in a few seconds. You will also notice that the default window is small. Enlarge the window and adjust column widths to see the information presented.

To help to focus your investigation you need filters. These help you to see what you are looking for rather than trawling through all this data. Type Ctrl+L to access the filter menu.

You should see something like the below. This shows that a number of filters are set by default to remove things which are unlikely to interest you. These include, for example, processes run by procmon.exe itself, Windows system processes and processes writing to \$MFT (MFT is the master file table) (and other files associated with NTFS).

Process Monitor Filter

Display entries matching these conditions:

Architecture is then Include

Reset Add Remove

| Column                                                                    | Relation    | Value         | Action  |
|---------------------------------------------------------------------------|-------------|---------------|---------|
| <input checked="" type="checkbox"/> <input type="checkbox"/> Process Name | is          | Procmon.exe   | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Process Name | is          | Procexp.exe   | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Process Name | is          | Autoruns.exe  | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Process Name | is          | Procmon64.exe | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Process Name | is          | Procexp64.exe | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Process Name | is          | System        | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Operation    | begins with | IRP_MJ_       | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Operation    | begins with | FASTIO_       | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Result       | begins with | FAST IO       | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Path         | ends with   | pagefile.sys  | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Path         | ends with   | \$Mft         | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Path         | ends with   | \$MftMirr     | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Path         | ends with   | \$LogFile     | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Path         | ends with   | \$Volume      | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Path         | ends with   | \$AttrDef     | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Path         | ends with   | \$Root        | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Path         | ends with   | \$Bitmap      | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Path         | ends with   | \$Boot        | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Path         | ends with   | \$BadClus     | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Path         | ends with   | \$Secure      | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Path         | ends with   | \$UpCase      | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Path         | contains    | \$Extend      | Exclude |
| <input checked="" type="checkbox"/> <input type="checkbox"/> Event Class  | is          | Profiling     | Exclude |

OK Cancel Apply

You should notice that, with these default filters in place, you see a small percentage of all activity. You can remove these filters to see all activities but, it is more likely that you will add more filters to focus on the specific activity of interest.

To start you need to focus on the Windows Registry. On the toolbar, disable all the application icons, apart from the Registry. Enter Ctrl + L to access the filter menu.

- In the first drop down box select Detail
- In the second drop down box select contains
- In the text enter box enter 3129
- In the third drop down box select Include
- Select Add
- Select OK

Start the capture process working with Ctrl +E.

Open the Windows Control Panel (Windows Key +R and type Control Panel)

Open Internet Options (under Network and Internet) from the Control Panel. Modify the proxy settings to be Port Number 3129 [you will also need to set an IP, use 170.0.0.1]. Click OK. [we could have used another port or IP, this is just an example].

After a few seconds stop capture (also Ctrl +E).

You should see some activity in Process Monitor. As you have filtered for 3129, Procmon has identified it when it was written to the Registry. Expand the Path and Detail columns to see exactly where it was written to and the value.

You may need to repeat this change a couple of times before you capture it correctly. You can use any Port Number not already in use.

As a general knowledge filler, it would be good to review what Port Numbers are and which are available on this link.

Remove your Proxy settings (otherwise, you won't be able to access the Internet).

NOTE: Procmon won't record anything when you change the number. This is because all values are filtered out except 3129.

Try some of the other filters - play with Process Monitor.

This is only a small example to illustrate the power of Procmon. The same method can be applied to view any system activity. Process Monitor can be used to monitor all changes on a Windows system, not just registry changes. It is invaluable when you come across an unknown software tool (including an unknown virus).

### Lab 8.3: Live Forensics

So you have reason to believe one of your servers has experienced a security compromise... What next? For this lab you perform some standard steps that you would perform on a real compromised running system.

The investigation of a potential security compromise is closely related to digital forensics topics. As with forensic investigations, we also aim to maintain the integrity of our "evidence", wherever possible not modifying access times or other information. However, in a business incident response setting maintaining a chain of evidence may not be our highest priority, since we may be more concerned with other business objectives, such as ensuring confidentiality, integrity, and availability of data and services.

During analysis, it is good practice to follow the order of volatility (OOV): collecting the most volatile evidence first (such as the contents of RAM, details of processes running, and so on) from a live system, then collecting less volatile evidence (such as data stored on disk) using offline analysis.

For ease we will continue with your Ubuntu VM, the one you created at the start of the lab.

You can make your Ubuntu VM have a better aspect ratio, and copy and paste to the host, by installing VMWare tools. Click on **Player -> Manage -> Install VMWare Tools**

Open *Computer* - it is on the desktop of your VM.

You should see an entry for "VMware Virtual SATA CDRW Drive"

Double click it

This is the mount point for the CD. And this is how you will be able to access its contents.

Note your mount point:

In terminal (ctrl+alt+t) Change your directory to /tmp:

```
cd /tmp
```

Make a temporary directory to store VMware Tools to.

```
mkdir VMWare
```

Change the directory to your newly created directory:

```
cd /VMWare
```

Now extract VMWare tools [replace mount point with your mount point]:

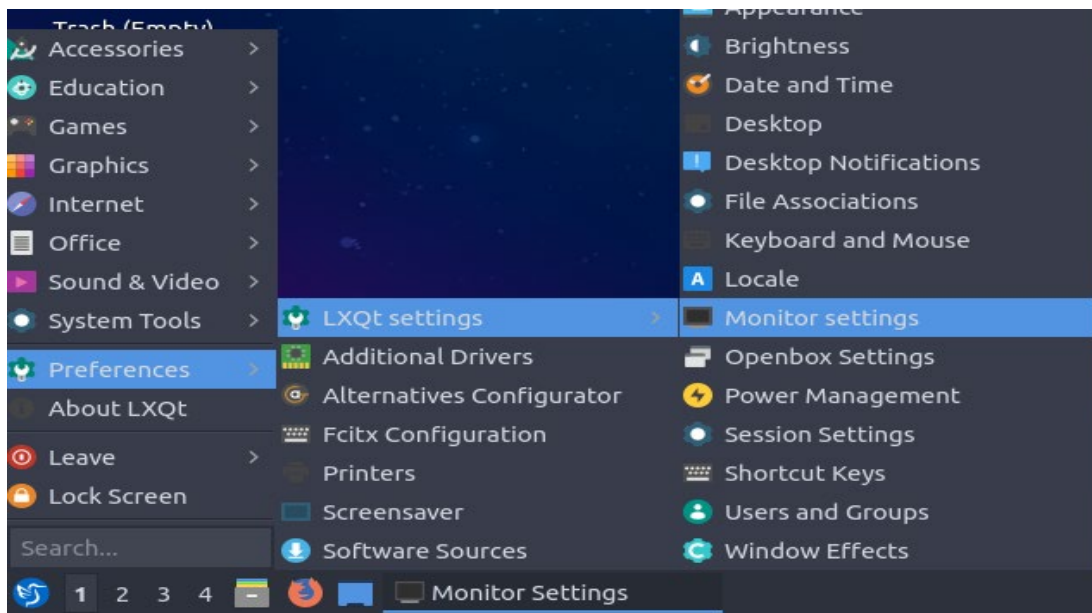
You can press tab and Linux will auto complete filenames. So in the below type up to the end of the / after your mount point and press tab twice it will list the contents of the folder. You can then start typing (capitalisation is important) and press tab to complete.

```
tar xpf MOUNTPOINT/VMwareTools-x.x.x-yyyy.tar.gz
```

The value x.x.x is the product version number, and yyyy is the build number of the product release.

Type Yes to the first question. You can then just press enter to the rest and it will take the default response.

Once installed, reboot your VM. And go to Monitor Settings:



Then you will be able to change the resolution to something more comfortable.

If you reboot your VM you should also have copy paste functionality between your host system (the physical machine) and the VM.

### Lab 8.3.1: Live Analysis

On your machine download F.I.R.E., which is a free Forensic and Incident Response Environment on a bootable CD-ROM. Download from this link:

<https://tinyurl.com/mpzm63tc>

It is an old system but is comparable to many of the systems still on the market. Add the ISO to the VM, see below.

- Select "Edit Virtual Machine Settings".
- Click "CD/DVD (IDE)".
- Select "Use ISO image file".
- Ensure connect at power on is selected.
- Click OK.

[if you get a warning about a drive lock, press OK to continue]

In the real world we would have a forensic system or a USB connected to the system we were analysing - maybe through the LAN. To replicate this we will use a USB drive.

After suspecting a compromise, before powering down the server for offline analysis, the first step is typically to perform some initial investigation of the live system. Live analysis aims to investigate suspicions that a compromise has occurred, and gather volatile information, which may potentially include evidence that would be lost by powering down the computer.



Before we begin let's ensure we know where our USB is mounted.

Open Computer - it is on the desktop of your VM.

You should see an entry for your USB

Double click it

Along the top bar you should see something like we did for the CD but this time for your USB - we will refer to this as USBMOUNT from now on:

This is the mount point for the USB. And this is how you will be able to access its contents.

**Note your USBMOUNT point.**

To keep a record of what we are doing on the system, start the script command through the terminal (under System Tools -> QTerminal or Ctrl + alt + t):

```
mkdir /USBMOUNT/Case001
```

```
Mkdir /USBMOUNT/Case001/Evidence
```

```
script -f /USBMOUNT/Case001/invst.log
```

Note: if you do this lab over multiple sessions, be sure to save the log of your progress (/USBMOUNT/Case001/Evidence/invst.log), and restart script.

**What does the Script command in Linux do?**

Many of the commands used to investigate what is happening on a system are standard Unix commands. However, it is advisable to run these from a read-only source, since software on your system may have been tampered with. Also, using read-only media minimises the changes made to your local filesystem, such as executable file access times.

During preparation, you configured the compromised VM to have access to the FIRE (Forensic and Incident Response Environment) CD/DVD ISO (which is equivalent to inserting the optical disk into your server's DVD-tray). FIRE is an example of a Linux Live Disk that includes tools for forensic investigation. In addition to being able to boot to a version of Linux for offline investigation of evidence, the disk contains Linux tools for live analysis.

On a typical system, many binary executables are dynamically linked; that is, these programs do not physically contain all the libraries (shared code) they use, and instead load that code from shared library files when the program runs.

On Unix systems the shared code is typically contained in ".so" files, while on Windows ".dll" files contain shared code. The risks associated with using dynamically linked executables to investigate security breaches is that access times on the shared objects will be updated, and the shared code may also have been tampered with. For this reason it is safest to use programs that are statically linked; that is, have been compiled to physically contain a copy of all of the shared code that it uses.

First let's record the path of the CD. This will vary depending on the username you used:

In the terminal look at which libraries are dynamically loaded when you run a typical command:

```
ldd /bin/ls
```

Examine the output, and determine how many external libraries are involved.

The FIRE disk contains a number of statically compiled programs to be used for investigations. Check that these are indeed statically linked (replace MOUNTPOINT with your mount point):

```
ldd MOUNTPOINT/statbins/linux2.2_x86/ls
```

Compare the output to the previous command run on your own system. The output will be distinctly different, stating that the program is not dynamically compiled.

Note that, although an improvement, using statically linked programs such as these still does not guarantee that you can trust the output of the programs you run. Consider why, and [make a note of this](#).

To save yourself from having to type `"/mnt/cdrom/statbins/linux2.2_x86/"` again and again, save that value in a Bash variable, you will need to replace the MOUNT POINT below with where FireBase is located, follow the same instructions as we did for VMWare Tools.:

```
export static="[MOUNT POINT]/statbins/linux2.2_x86/"
```

Now, to run the statically compiled version of ls, you can run [type it exactly as shown]:

```
$static/ls
```

### Lab 8.3.2: Collecting Live State Manually

The next step is to use tools to capture information about the live system, for later analysis.

#### *Comparing process lists*

We can send a list of processes to our evidence store using:

```
ps aux > /USBMOUNT/Case001/Evidence/ps_out
```

Run the statically compiled version of ls from the incident response disk to list the contents of /proc (this is provided dynamically by the kernel: a directory exists for every process on the system).

Run the command:

```
$static/ls /proc > /USBMOUNT/Case001/Evidence/proc_ls_static"
```

Find the newly created files and compare the list of pids (numbers representing processes) output from the previous commands. This is the second column of output in the ps\_out, with the numbers in proc\_ls\_static.

Are the same processes shown each time? If not, that is very suspicious, and likely indicates a break-in, and that we probably shouldn't trust the output of local commands.

### *Gathering live state using statically compiled programs*

Each of the commands given below will allow you to capture information from a running system. This list is not exhaustive. You should look at each command and research what it does and what each of the flags means.

Save a copy of a list of deleted files that are still executing - the system may have just automatically deleted them but they are still actively running [this comes from The Sleuth Kit (TSK)]:

```
sudo $static/ils -o /dev/sda1 > /USBMOUNT/Case001/Evidence/deleted_out
```

Save a list of the files currently being accessed by programs:

```
sudo $static/lsof > /USBMOUNT/Case001/Evidence/lsof_out
```

Save a list of network connections:

```
sudo $static/netstat -a > /USBMOUNT/Case001/Evidence/netstat_out
```

(Some commands such as this one may take awhile to run, wait until the Bash prompt returns)

Save a list of the network resources currently being accessed by programs:

```
sudo $static/lsof -P -i -n > /USBMOUNT/Case001/Evidence/lsof_net_out
```

Save a copy of the routing table:

```
sudo $static/route > /USBMOUNT/Case001/Evidence/route_out
```

Save a copy of the ARP cache:

```
sudo $static/arp -a > /USBMOUNT/Case001/Evidence/arp_out
```

Save a list of the kernel modules currently loaded (as reported by the kernel):

```
sudo $static/cat /proc/modules > /USBMOUNT/Case001/Evidence/lsm_out
```

### *Creating images of the system state*

We can take a snapshot of the live state of the computer by dumping the entire contents of memory (what is in RAM/swap) into a file. On a Linux system /proc/kcore contains an ELF-formatted core dump of the kernel. Save a snapshot of the kernel state:

```
sudo $static/dd if=/proc/kcore conv=noerror,sync > /USBMOUNT/Case001/Evidence/kcore
```

Next, we can copy entire partitions, to preserve the exact state of stored data, and so that we can conduct offline analysis without modifying the filesystem.

Start by identifying the device files for the partitions on the compromised system:

```
df
```

Note that on this system the root partition (mounted on "/"), is /dev/sda1.

List all the files you have created:

```
ls -la /USBMOUNT/Case001/Evidence/
```

At this stage we are not interested in the analysis of the data. This is about data preservation. We can now take the system offline, or run an image to a remote device - using ssh or something similar.

### Lab 8.3.3: Collecting Live State Using Scripts

As you may have concluded from the previous tasks, manually collecting all this information from a live system can be a fairly time consuming process. Incident response data collection scripts can automate much of this process. A common data collection script "linux-ir.sh", is included on the FIRE disk, and is also found on the popular Helix IR disk.

Have a look through the script:

```
less $static/linux-ir.sh
```

Note that this is a Bash script, and each line contains commands that you could type into the Bash shell. Bash provides the command prompt on most Unix systems, and a Bash script is an automated way of running commands. This script is quite simple, with a series of commands (similar to some of those you have already run) to display information about the running system.

Identify some commands within the script that collect information you have not already collected above.

Exit viewing the script (press q).

Run the data collection script, redirecting output to your openSUSE VM:

```
$static/linux-ir.sh > /USBMOUNT/Case001/Evidence/ir_out
```

On your openSUSE VM, have a look at the output from the script:

```
less /USBMOUNT/Case001/Evidence/ir_out
```

Use what you have learnt to spot some evidence of a security compromise.

### Lab 8.3.4: Checking for Rootkits

An important concern when investigating an incident, is that the system (including user-space programs, libraries, and possibly even the OS kernel) may have been modified to hide the presence of changes made by an attacker. For example, the ps and ls commands may be modified, so that

certain processes and files (respectively) are not displayed. The libraries used by various commands may have been modified, so that any programs using those libraries are provided with deceptive information. If the kernel has been modified, it can essentially change the behaviour of any program on the system, by changing the kernel's response to instructions from processes. For example, if a program attempts to open a file for viewing, the kernel could provide one set of content, while an attempt to execute the file may result in a completely different program running.

Detecting the presence of rootkits is tricky, and prone to error. However, there are a number of techniques that, while not foolproof, can detect a number of rootkits. Methods of detection include: looking for inconsistencies between different ways of gathering data about the system state, and looking for known instances of malicious files.

Chkrootkit is a Bash script that performs a number of tests for the presence of various rootkits.

Have a quick look through the script, it is much more complex than the previous linux-ir.sh script:

```
less $static/chkrootkit
```

Exit less

Confirm that if we were to run ls, we would be running the local (dynamic) version, probably /bin/ls:

which ls

To understand why, look at the value of the environment variable \$PATH, which tells Bash where to look for programs:

```
echo $PATH
```

Set the \$PATH environment variable to use our static binaries wherever possible, so that when chkrootkit calls external programs it will (wherever possible) use the ones stored on the IR disk:

```
export PATH=$static:$PATH
```

Confirm that now if we were to run less, we would be running the static version:

which ls

This should report the path to our static binary on the FIRE disk.

It is now safe to run chkrootkit :

```
./chkrootkit-linux/chkrootkit > /USBMOUNT/Case001/Evidence/chkrootkit_out
```

Help: you may get a message in the terminal before you type the password. You should still type the password for the script to run. The script should not take long to run.

Have a look at the output:

```
less /USBMOUNT/Case001/Evidence/chkrootkit_out
```

From the output, identify files or directories reported as "INFECTED", or suspicious.

Can you find your `.bash_history` file and export this to your evidence drive?

Power down the system, so that we can continue analysis offline:

```
$static/sync; $static/sync
```

If you do not know what the `sync` command does, run “info coreutils 'sync invocation'” for more information.

Player -> Power -> Shutdown Guest

Why might we want to force a power off (effectively “pulling the plug”), rather than going through the normal shutdown process (by running “halt” or “shutdown”)?

### Lab 8.3.5: Offline Analysis of Live Data Collection

We can recover commands that were run the last time the computer was running. This is possible by searching through the saved RAM (the `kcore` ELF dump we saved earlier).

You should find your USB is now back available in your host operating system. Access Bintext from the *lab08 resources folder*. This will allow us to extract ASCII strings from the binary core dump.

Using Bintext add the `kcore` file you saved previously.

Look for evidence of the commands you ran before saving the kernel core dump.

### Lab 8.3.6: Windows and Live Incident Response

The theory you have applied in this lab applies similarly to other operating systems such as Windows. Many incident response live disks (such as FIRE and Helix) also include Windows programs that can collect information about the state of a system.

**Unguided task:** use an incident response disk (such as Helix), can be downloaded from this link:

<https://tinyurl.com/4n7f65yh>

to obtain information from a running Windows computer. You should collect a dump of the contents of RAM, and a list of running processes and network connections.

Write a paragraph detailing what you expect to find from the Windows system. After your live analysis, make a list of what you recovered from the contents of RAM, the list of running processes and network connections. Did you find any content, which might be of interest to an investigation?

**End of Lab Sheet**